

LiveKit-Videointegration für RailTime — Detaillierter Integrationsplan

Projekt	RailTime (internes Mitarbeiterportal, Bahnlogistik)
Dokuments tand	28.07.2026
Branch	claude/livekit-video-implementation-jadpwc
Zielgruppe	Entwickler:innen und LLM-gestützte Implementierung
Quellen	Dieses Dokument existiert als LIVEKIT_INTEGRATION_PLAN.md (maschinenlesbare Referenz) und LIVEKIT_INTEGRATION_PLAN.pdf (identischer Inhalt) im Repo-Grundverzeichnis

Zweck: Vollausbau-Videotelefonie mit LiveKit als Media-Server (SFU), Laravel Reverb ausschließlich für Signalisierung und Benachrichtigungen, coturn als Firewall-Fallback. Dieses Dokument übersetzt den 8-Phasen-Fahrplan in konkrete, projektspezifische Arbeitsschritte mit echten Dateipfaden, Klassennamen, Ports und Konfigurationen.

> **Umsetzungsstand (28.07.2026):** Die App-seitigen Phasen 4–7 (Raumverwaltung, Tokens/Beitritt, Einladungssystem, Live-Steuerung/Moderation) sind auf diesem Branch **bereits implementiert** — Migrationen, Models, Services (app/Services/Calls/), Events, Token-/Webhook-Controller, Livewire-UI (CallWindow, IncomingCallOverlay), resources/js/calls.js und die Artisan-Kommandos railtime:livekit-keys / railtime:livekit-check liegen im Repository; die Testsuite (inkl. tests/Feature/CallFlowTest.php) ist grün. **Offen sind die Server-Phasen 1–3 und 8** — die Schritt-für-Schritt-Anleitung dafür steht in SERVER_SETUP.md (+ PDF) im Grundverzeichnis.

Inhaltsverzeichnis

1. Ist-Zustand (verifiziert am Codebestand)	3
2. Architekturentscheidungen (vorab zu bestätigen)	3
3. Zielarchitektur	4
DNS- und Firewall-Matrix	5
4. Phase 1 — Grundinfrastruktur	5
5. Phase 2 — coturn (Firewall-Fallback)	5
6. Phase 3 — LiveKit-Media-Server	6
6.1 livekit.yaml	6
6.2 docker-compose.yml	7
6.3 systemd-Unit /etc/systemd/system/railtime-media.service	7
6.4 Artisan-Werkzeuge (im Repo, nach Reverb-Vorbild)	8
7. Phase 4 — Raumverwaltung in Laravel	8
7.1 Migrationen (Namenskonvention des Repos)	8
7.2 Models	9
7.3 RBAC-Anbindung (bestehendes System, kein neues Framework)	9
7.4 Lifecycle-Service	10
8. Phase 5 — Tokens und Beitritt	10
8.1 Pakete	10

8.2 app/Services/Calls/LiveKitService.php	10
8.3 Token-Endpoint (bewusst klassischer Controller, nicht Livewire)	10
8.4 Konfiguration	11
8.5 Frontend	11
9. Phase 6 — Einladungssystem	11
9.1 Ablauf Ende-zu-Ende	11
10. Phase 7 — Live-Steuerung über Reverb	12
10.1 Channel	12
10.2 Events (alle nach ChatMessageSent-Vorbild: ShouldBroadcastNow + broadcastAs)	12
10.3 LiveKit-Webhooks (DB-Wahrheit)	13
10.4 Moderation (server-autoritativ)	13
11. Phase 8 — Test und Absicherung	13
11.1 Testmatrix	13
11.2 Betrieb & Monitoring	14
11.3 Risiken	14
12. Aufwand und Umsetzungsreihenfolge	14
Anhang A — Datei-für-Datei-Änderungsliste (Referenz für die Implementierung)	15
Anhang B — Plan B: LiveKit auf dem Plesk-Host (falls keine zweite VM)	16
Anhang C — Umgebungsvariablen (Übersicht)	16

1. Ist-Zustand (verifiziert am Codebestand)

Diese Fakten wurden direkt aus dem Repository erhoben und sind Grundlage aller folgenden Entscheidungen:

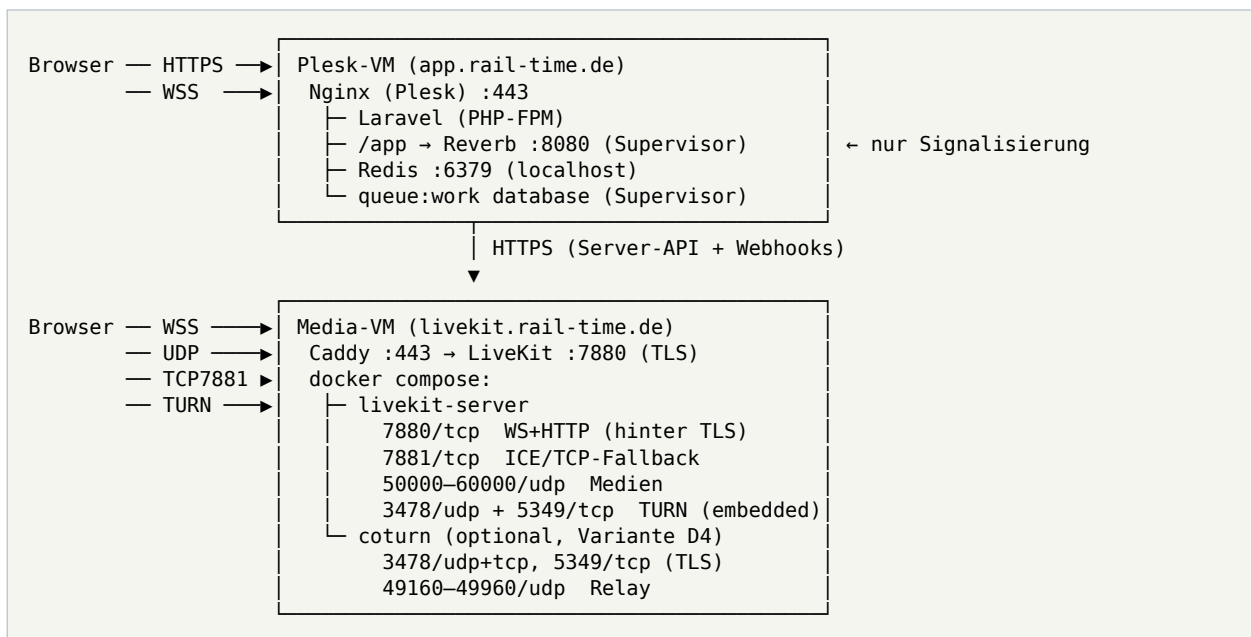
- **Stack:** Laravel 12, PHP ^8.2, Jetstream ^5.5 (Livewire-3-Stack, Teams aktiviert), Sanctum. Frontend: Blade + Livewire 3 + Alpine 3 + Tailwind, Vite 6. Kein Vue/React/Inertia.
- **Reverb ^1.10 läuft bereits** (Port 8080 hinter manuell konfiguriertem Plesk/Nginx-WSS-Proxy). Echo + pusher-js sind in `resources/js/app.js` (ca. Zeile 84–113) initialisiert. `BROADCAST_DRIVER=reverb`.
- **Broadcast-Muster vorhanden:** Events in `app/Events/` (z. B. `ChatMessageSent`) implementieren `ShouldBroadcastNow` mit `broadcastAs`-Namen wie `chat.message.sent`. Channels in `routes/channels.php`: `App.Models.User.{id}` und `chat.{chatId}`.
- **Rechteverwaltung ist komplett custom** (kein spatie/laravel-permission):
- Globale Rolle: Spalte `users.role` (`admin/staff`), Middleware-Alias `role`, `User::isAdmin()`.
- Team-RBAC: JSON-Spalte `teams.rbac_permissions`, Katalog `app/Support/Rbac/RbacCatalog.php`, Prüfung `User::hasRbacPermission()`. Gates werden in `app/Providers/AuthServiceProvider.php` automatisch pro Katalog-Key registriert (inkl. `Gate::before-Admin-Bypass`). Admin-UI: `app/Livewire/Admin/Employees/TeamRbacModal.php`. **Neue Keys im Katalog erzeugen Gate + UI automatisch.**
- **Chat vorhanden:** `app/Models/Chat.php` (`type direct|group`, `directBetween()`, `participants()`, `canManageGroup()`), `app/Livewire/ChatBox.php`, Pivot `chat_user`. Natürlicher Anker für „Anruf in diesem Chat“.
- **Web-Push vorhanden:** `laravel-notification-channels/webpush`, `app/Support/Push/` mit `PushCategory` (Enum, aktuell `messages`, `chat`), `PushDelivery` (Deep-Link-Helfer), Models `PushSubscription/NotificationPreference`, `User::wantsWebPush()`.
- **Sound-System vorhanden:** `app/Support/Sound/SoundLibrary.php` + `resources/js/realtime-notification-sound.js`. Cross-Tab-Koordination über `BroadcastChannel('railtime-app-presence')` in `resources/js/notification-presentation.js`.
- **Deployment: Plesk, kein Docker im Repo.** Vorbild-Muster: `php artisan railtime:reverb-keys` (`app/Console/Commands/GenerateReverbCredentials.php`) und `railtime:install-reverb-service` (Supervisor-Installer). `QUEUE_CONNECTION=database`, Redis konfiguriert aber ungenutzt, `routes/api.php` leer.
- **Kein WebRTC-/Video-Code vorhanden** — Greenfield. Vorhandene `getUserMedia/MediaRecorder`-Nutzung (Sprachnachrichten) belegt, dass die Berechtigungs-UX im Frontend bereits funktioniert.
- Sprache: UI/Doku Deutsch, Klassennamen Englisch, Zeitzone Europe/Berlin.

2. Architekturentscheidungen (vorab zu bestätigen)

#	Entscheidung	Empfehlung	Begründung	
D1	LiveKit Cloud vs. self-hosted	Self-hosted	Datensouveränität/DSGVO für ein internes HR-nahes Portal. Der Laravel-Code ist identisch — bei Cloud ändern sich nur <code>LIVEKIT_URL</code> und die Keys. Entscheidung ist reversibel.	

#	Entscheidung	Empfehlung	Begründung	
D2	Media-Host: gleicher Server wie Plesk vs. eigene VM	Eigene kleine Media-VM (2 vCPU / 4 GB RAM, z. B. Hetzner CX32, ~6-10 €/Monat) mit Docker Engine	Plesk beansprucht Port 443/80 und kann eigene Nginx-Configs überschreiben; der Media-Server braucht rohe UDP-Ranges und idealerweise ein eigenes :443 für TURN-TLS. Der Plesk-Host und das Repo bleiben Docker-frei. Plan B „gleicher Host“ siehe Anhang B.	
D3	Externe Gäste (Nicht-Nutzer) einladen	v2, nicht v1	Schema ist vorbereitet (nullable user_id + guest_name bei Teilnehmern, tokenbasierter Join), aber keine Gast-UI in v1. Halbiert die Auth-Komplexität.	
D4	Eingebautes LiveKit-TURN vs. externes coturn	Start mit eingebautem TURN, coturn als Härtingsstufe im selben Compose-File	Das eingebaute TURN verwaltet Zugangsdaten pro Session automatisch (kein statisches Secret, das leaken kann). coturn bleibt wie geplant Bestandteil von Phase 2 und wird über <code>LIVEKIT_TURN_MODE=embedded</code>	coturn` umschaltbar. Entscheidend ist in beiden Varianten ein TURN-TLS-Listener, der auf 443 erreichbar ist.

3. Zielarchitektur



Verantwortungstrennung (verbindlich):

Ebene	Zuständig für	Nie zuständig für
Reverb	Klingeln, Einladungs-Antworten, „Anruf läuft“-Banner, Moderations-Feedback an die UI	Medien, In-Raum-Teilnehmerstatus
LiveKit-Client-Events	Alles im Raum: Teilnehmer join/leave, Tracks, Sprecher-Erkennung, Verbindungsqualität	— (nicht über Reverb spiegeln — doppelte Quellen erzeugen Ordering-Bugs)
LiveKit-Webhooks → Laravel	DB-Wahrheit: Raumstatus, Teilnahme-Zeitstempel (selbstheilend bei Tab-Crash)	UI-Echtzeit (dafür zu langsam)

DNS- und Firewall-Matrix

DNS	Zeigt auf	Eingehende Ports
app.rail-time.de	Plesk-VM	443/tcp (besteht; Reverb-WSS darunter proxied)
livekit.rail-time.de	Media-VM	443/tcp (TLS→7880), 7881/tcp, 50000–60000/udp
turn.rail-time.de	Media-VM (zweite IP oder SNI)	3478/udp+tcp, 5349/tcp (TURN-TLS), möglichst 443/tcp; Relay 49160–49960/udp

Firewall-Traversal-Leiter (für strenge Firmennetze, Reihenfolge des automatischen Fallbacks):

1. UDP 50000–60000 (Normalfall, beste Qualität)
2. TCP 7881 (ICE über TCP)
3. TURN über UDP 3478
4. **TURN über TLS 5349 bzw. 443** — funktioniert fast überall, da von HTTPS nicht unterscheidbar

4. Phase 1 — Grundinfrastruktur

Es ist nichts zu bauen — nur festzuschreiben und zu prüfen.

Festlegung: Reverb übernimmt ausschließlich Signalisierung und Benachrichtigungen, niemals Medien. Der Plesk-Nginx proxied niemals Medienströme — nur den bestehenden Reverb-WSS-Pfad.

Prüfliste:

- ☐ Reverb über WSS von extern erreichbar (bestehender Plesk-Proxy), Zertifikat gültig.
- ☐ Queue-Worker (`QUEUE_CONNECTION=database`, Plesk-Supervisor) läuft — er trägt später die Ring-Timeout-Jobs. Latenz der Database-Queue ($\pm$ Sekunden) ist für 45-s-Timeouts akzeptabel.
- ☐ Entscheidungen D1–D4 (Abschnitt 2) bestätigt; bei D2 „eigene VM“: VM bestellt, DNS-Einträge `livekit.` und `turn.` angelegt.

Aufwand: **0,5 Tage**

5. Phase 2 — coturn (Firewall-Fallback)

Auf der Media-VM als Teil des Compose-Stacks (siehe Phase 3). Konfiguration

`/opt/railtime-media/turnserver.conf`:

```

listening-port=3478
tls-listening-port=5349
realm=turn.rail-time.de
fingerprint

# Statische Zugangsdaten (wie im Fahrplan vorgesehen):
lt-cred-mech
user=railtime:<langes-statisches-passwort>
# Empfohlene Alternative für Langzeitbetrieb: HMAC-zeitbegrenzte Credentials,
# die Laravel pro Sitzung ausstellt (weniger Leak-Risiko):
#   use-auth-secret
#   static-auth-secret=<secret>

min-port=49160
max-port=49960
cert=/etc/letsencrypt/live/turn.rail-time.de/fullchain.pem
pkey=/etc/letsencrypt/live/turn.rail-time.de/privkey.pem

# Härtung: Relay in private Netze verbieten
no-multicast-peers
denied-peer-ip=10.0.0.0-10.255.255.255
denied-peer-ip=172.16.0.0-172.31.255.255
denied-peer-ip=192.168.0.0-192.168.255.255

```

Arbeitsschritte:

1. DNS `turn.rail-time.de` → Media-VM (idealerweise zweite IP, damit TURN-TLS auf 443 lauschen kann).
2. Let's-Encrypt-Zertifikat für `turn.rail-time.de` (certbot standalone oder DNS-Challenge; Renewal-Hook: coturn-Container neu starten).
3. Firewall: 3478/udp+tcp, 5349/tcp, 49160-49960/udp öffnen.
4. Test: `turnutils_uclient -T -u railtime -w <passwort> turn.rail-time.de` bzw. Trickle-ICE-Testseite mit `turns:turn.rail-time.de:5349`.

Hinweis zu D4: In der Startkonfiguration ist das **eingebaute LiveKit-TURN aktiv** und coturn im Compose-File vorbereitet, aber gestoppt. Umschalten erfolgt über `LIVEKIT_TURN_MODE=coturn` in der Laravel-.env — dann liefert der Token-Endpoint die coturn-ICE-Server an die Clients aus (Abschnitt 8).

Aufwand: **1 Tag**

6. Phase 3 — LiveKit-Media-Server

Alles auf der Media-VM unter `/opt/railtime-media/`. **Docker nur dort — das Repo und der Plesk-Host bleiben Docker-frei.**

6.1 livekit.yaml

```

port: 7880
bind_addresses: ["0.0.0.0"]
rtc:
  udp_port_range_start: 50000
  udp_port_range_end: 60000
  tcp_port: 7881
  use_external_ip: true          # öffentliche IP per STUN ermitteln
keys:
  RAILTIME_API_KEY: "<api-secret>"    # erzeugen: livekit-server generate-keys
                                      # bzw. php artisan railtime:livekit-keys (6.4)
webhook:
  api_key: RAILTIME_API_KEY
  urls:
    - https://app.rail-time.de/webhooks/livekit
turn:
  # Variante A (Start): eingebautes TURN
  enabled: true
  domain: turn.rail-time.de
  tls_port: 5349                # ideal: 443 auf dedizierter IP
  udp_port: 3478
  external_tls: true            # Caddy terminiert TLS davor
logging:
  level: info

```

6.2 docker-compose.yml

```

services:
  livekit:
    image: livekit/livekit-server:v1.8
    network_mode: host          # zwingend: 10.000 UDP-Ports sind per Port-Mapping unbrauchbar
    volumes:
      - ./livekit.yaml:/etc/livekit.yaml:ro
    command: --config /etc/livekit.yaml
    restart: unless-stopped

  caddy:
    # TLS für 7880 (WSS) und 5349 (TURN-TLS)
    image: caddy:2
    network_mode: host
    volumes:
      - ./Caddyfile:/etc/caddy/Caddyfile:ro
      - caddy-data:/data
    restart: unless-stopped

  coturn:
    # Variante B (D4) – vorbereitet, Start nur bei Bedarf
    image: coturn/coturn:latest
    network_mode: host
    volumes:
      - ./turnserver.conf:/etc/coturn/turnserver.conf:ro
      - /etc/letsencrypt:/etc/letsencrypt:ro
    profiles: ["coturn"]        # docker compose --profile coturn up -d
    restart: unless-stopped

volumes:
  caddy-data:

```

Caddyfile:

```

livekit.rail-time.de {
  reverse_proxy 127.0.0.1:7880
}

```

6.3 systemd-Unit /etc/systemd/system/railtime-media.service

```
[Unit]
Description=RailTime Media-Stack (LiveKit + Caddy [+ coturn])
After=docker.service network-online.target
Requires=docker.service

[Service]
Type=oneshot
RemainAfterExit=yes
WorkingDirectory=/opt/railtime-media
ExecStart=/usr/bin/docker compose up -d
ExecStop=/usr/bin/docker compose down

[Install]
WantedBy=multi-user.target
```

6.4 Artisan-Werkzeuge (im Repo, nach Reverb-Vorbild)

Command	Datei	Zweck
railtime:livekit-keys	app/Console/Commands/GenerateLiveKitCredentials.php	API-Key/Secret-Paar erzeugen, in .env schreiben, keys:-Block für livekit.yaml ausgeben (Spiegel von GenerateReverbCredentials)
railtime:livekit-check	app/Console/Commands/CheckLiveKitConnection.php	Server-API anpingen (listRooms), Webhook-Erreichbarkeit prüfen, aktive ICE-Konfiguration ausgeben; läuft zusätzlich im Scheduler mit Fehlerbenachrichtigung

Abnahme Phase 3: railtime:livekit-check grün; Browser-Probe `wss://livekit.rail-time.de` verbindet; Webhook-Test-Event kommt in Laravel an.

Aufwand: **1-1,5 Tage**

7. Phase 4 — Raumverwaltung in Laravel

7.1 Migrationen (Namenskonvention des Repos)

database/migrations/2026_07_29_000001_create_rooms_table.php

Spalte	Typ	Bemerkung			
id	bigIncrements				
uuid	uuid, unique	öffentlicher Bezeichner = LiveKit-Raumname			
name	string	Anzeigename			
type	enum: direct\	group\	meeting	direct/group ← aus Chat, meeting = frei/geplant	
status	enum: pending\	active\	ended\	cancelled, default pending	
owner_id	FK → users, cascadeOnDelete				
chat_id	FK → chats, nullable, nullOnDelete	Anker „Anruf im Chat“			
team_id	FK → teams, nullable, nullOnDelete				
scheduled_at / started_at / ended_at	timestamp nullable				

Spalte	Typ	Bemerkung			
settings	json nullable	z. B. {"video":true,"max_participants":15}			
timestamps		Indizes: (status), (chat_id)			

2026_07_29_000002_create_room_participants_table.php

Spalte	Typ	Bemerkung			
id, room_id FK cascade					
user_id	FK nullable	null = Gast (D3, v2)			
guest_name	string nullable	v2			
role	enum: host\	moderator\	speaker \	viewer, default speaker	
connection	enum: invited\	joined\	left\	disconnected, default invited	
livekit_identity	string	user-{id} — Abgleich mit Webhooks			
joined_at / left_at	timestamp nullable	unique (room_id, user_id)			

2026_07_29_000003_create_room_invitations_table.php

Spalte	Typ	Bemerkung				
id, room_id FK cascade						
inviter_id / invitee_id	FK → users, cascade					
status	enum: pending\	accepted\	declined\	expired \	missed, default pending	
expires_at	timestamp	Ring-Timeout, now()+45s				
responded_at	timestamp nullable	Index: (invitee_id, status)				

7.2 Models

- `app/Models/Room.php` — UUID-Erzeugung in `booted()::creating`; Casts `settings => array`, Zeitstempel `datetime`; Relations `owner()`, `chat()`, `team()`, `participants()`, `invitations()`; Helfer `isActive()`, `livekitRoomName()` (= uuid), `participantFor(User $u): ?RoomParticipant`, `canModerate(User $u): bool` (Owner v Teilnehmerrolle `host/moderator` v `hasRbacPermission('calls.moderate')`); Admin-Bypass kommt über `Gate::before()`.
- `app/Models/RoomParticipant.php`, `app/Models/RoomInvitation.php` — Standard-Relations + Status-Helfer.
- **Erweiterungen:** `User::roomParticipations()`, `User::activeCall(): ?Room`; `Chat::activeRoom(): ?Room` (`hasOne(Room::class)->whereIn('status', ['pending','active'])->latest()`) — damit zeigt die ChatBox „Anruf läuft — beitreten“.

7.3 RBAC-Anbindung (bestehendes System, kein neues Framework)

In `app/Support/Rbac/RbacCatalog.php` → `permissionGroups()` neue Gruppe:

```
'Anrufe' => [
    ['key' => 'calls.start', 'label' => 'Videoanrufe starten'],
    ['key' => 'calls.join', 'label' => 'Videoanrufen beitreten'],
    ['key' => 'calls.moderate', 'label' => 'Videoanrufe moderieren'],
],
```

Gates und der Eintrag im Team-RBAC-Modal entstehen **automatisch** über die bestehende Registrierung in AuthServiceProvider (inkl. Admin-Bypass). Bewusst kein `calls.invite.external` in v1 (D3) — der Key kann später einfach ergänzt werden.

7.4 Lifecycle-Service

`app/Services/Calls/RoomLifecycleService.php` — einziger Schreiber für `rooms.status` (aufgerufen von Livewire-Aktionen **und** Webhooks), alle Übergänge idempotent (`if ($room->status === 'ended') return;`). Methoden: `createForChat(Chat $chat, User $owner): Room`, `markActive()`, `markEnded(string $reason)`, `cancel()`.

Aufwand: **1,5 Tage**

8. Phase 5 — Tokens und Beitritt

8.1 Pakete

Seite	Paket	Begründung
Server	<code>composer require agenc104/livekit-server-sdk</code>	Statt handgerolltem JWT: liefert zusätzlich <code>RoomServiceClient</code> (Moderation, Phase 7) und <code>WebhookReceiver</code> (Signaturprüfung) — nutzt intern ohnehin <code>firebase/php-jwt</code> . Gekapselt hinter eigenem Service, damit der Vendor austauschbar bleibt.
Browser	<code>npm i livekit-client (^2)</code>	Reines TS/JS, passt exakt zum Alpine-Stack. Nicht <code>@livekit/components-react</code> .

8.2 `app/Services/Calls/LiveKitService.php`

Kapselt Token-Erzeugung und Server-API. Grant-Mapping (RBAC/Teilnehmerrolle → LiveKit VideoGrants):

Bedingung	VideoGrant
Gültige Einladung/Teilnahme + Gate <code>calls.join</code>	<code>roomJoin: true, room: {uuid}, canSubscribe: true, canPublishData: true</code> (Reaktionen/Chat über DataChannel)
Rolle speaker/host/moderator	+ <code>canPublish: true (canPublishSources: [camera, microphone, screen_share])</code>
Rolle viewer („nur zuhören“)	<code>canPublish: false</code>
Gate <code>calls.moderate</code> v Rolle host/moderator	+ <code>roomAdmin: true</code>
Gate <code>calls.start</code>	nur serverseitig: berechtigt zum Anlegen des Room-Datensatzes. Niemals <code>roomCreate</code> in Browser-Tokens — Räume erzeugt ausschließlich Laravel über die Server-API, Clients können keine Räume erfinden.

Token-Parameter: TTL 6 h, `identity: "user-{$user->id}"`, `name: $user->name`, `metadata: {"role": "<rolle>"}` (damit Clients Rollen ohne Zusatzrequest rendern können).

8.3 Token-Endpoint (bewusst klassischer Controller, nicht Livewire)

```
// routes/web.php, innerhalb der bestehenden Auth-Gruppe
// ['auth:sanctum', 'auth.status', 'jetstream.auth_session', 'verified']
Route::post('/calls/{room:uuid}/token', [CallTokenController::class, 'store'])
->middleware('throttle:20,1')->name('calls.token');
Route::get('/calls/{room:uuid}', CallWindow::class)->name('calls.window');
```

app/Http/Controllers/Calls/CallTokenController.php: autorisiert über Gate::authorize('calls.join') + Teilnahme-/Einladungsprüfung am Room; Antwort {token, wsUrl, iceServers}. Begründung Controller statt Livewire: das JS holt den Token per fetch() unmittelbar vor room.connect() — immer frisch, nie im Livewire-Snapshot serialisiert. iceServers enthält bei LIVEKIT_TURN_MODE=coturn die coturn-Einträge, sonst leer (eingebautes TURN wird von LiveKit selbst ausgehandelt).

8.4 Konfiguration

config/livekit.php (neu):

```
return [
    'url'          => env('LIVEKIT_URL', 'https://livekit.rail-time.de'), // Server-API (Twirp)
    'ws_url'       => env('LIVEKIT_WS_URL', 'wss://livekit.rail-time.de'), // Browser
    'api_key'      => env('LIVEKIT_API_KEY'),
    'api_secret'   => env('LIVEKIT_API_SECRET'),
    'token_ttl'    => env('LIVEKIT_TOKEN_TTL', 21600),
    'ring_timeout' => env('CALL_RING_TIMEOUT', 45),
    'turn' => [
        'mode'      => env('LIVEKIT_TURN_MODE', 'embedded'), // embedded|coturn
        'url'       => env('TURN_URL'), // turns:turn.rail-time.de:5349
        'username'  => env('TURN_USERNAME'), // nur bei coturn/static
        'credential' => env('TURN_CREDENTIAL'),
    ],
];
```

.env.example-Ergänzungen: LIVEKIT_URL, LIVEKIT_WS_URL, LIVEKIT_API_KEY, LIVEKIT_API_SECRET, LIVEKIT_TOKEN_TTL=21600, CALL_RING_TIMEOUT=45, LIVEKIT_TURN_MODE=embedded, TURN_URL=, TURN_USERNAME=, TURN_CREDENTIAL=.

Bewusst kein VITE_LIVEKIT_*: wsUrl und iceServers kommen zur Laufzeit aus dem Token-Endpoint — Infrastruktur-Änderungen erfordern damit keinen Frontend-Rebuild.

8.5 Frontend

- app/Livewire/Calls/CallWindow.php + View: Vollseiten-Komponente (Route oben), rendert das Video-Grid-Gerüst und hält die Management-Aktionen (endCall, muteParticipant, removeParticipant, toggleRole) — alle Gate-geprüft. Die eigentliche Medienlogik macht Alpine.
- resources/js/calls.js: Alpine-Komponente Alpine.data('callRoom', ...) um livekit-client — connect, Track-Attach/Detach ans DOM-Grid, Mikro/Kamera/Screenshare-Toggles, Active-Speaker, RoomEvent.ParticipantConnected/Disconnected, Reconnect-Handling.
- **Vite:** resources/js/calls.js als **eigener Input** in vite.config.js; Laden nur in der CallWindow-View via @vite('resources/js/calls.js') — hält die ~80 KB (gz) livekit-client aus dem globalen app.js-Bundle. Die leichte Klingel-Overlay-Logik (ohne livekit-client-Import) wandert in die Nachbarschaft von app.js/notification-presentation.js.

Aufwand: **2-3 Tage**

9. Phase 6 — Einladungssystem

9.1 Ablauf Ende-zu-Ende

1. **Anrufer** klickt den Anruf-Button in der ChatBox → Livewire-Aktion → Gate::authorize('calls.start') → RoomLifecycleService::createForChat() (DB-Room, Teilnehmerzeile role=host; parallel LiveKitService::createRoom() mit empty_timeout: 120, max_participants).

2. `app/Services/Calls/CallInvitationService.php` → `invite(Room $room, Collection $invitees): pro` Empfänger mit `calls.join`:
 - DB-Zeile `room_invitations` (`expires_at = now()->addSeconds(config('livekit.ring_timeout'))`),
 - Teilnehmerzeile `connection=invited`,
 - Broadcast `CallInvitationSent` auf `App.Models.User.{id}`,
 - Web-Push `app/Notifications/IncomingCallNotification.php` mit neuem `PushCategory::Calls` (Case `Calls = 'calls' + Lang-Key` ergänzen), `urgency: high`, **TTL = Ring-Fenster** (ein nach dem Klingeln zugestellter Push ist schlimmer als keiner), Deep-Link `/calls/{uuid}` über die bestehenden `PushDelivery-Helfer`.
3. Dispatch `app/Jobs/ExpireCallInvitation.php` (`database-Queue, ->delay($expiresAt)`): falls noch pending → status missed, Broadcast `CallInvitationExpired`, Hinweis „Verpasster Anruf“.
4. **Empfänger:** `app/Livewire/Calls/IncomingCallOverlay.php` (einmal im Master-Layout gemountet, wie die bestehende Notification-UI empfängt `call.invited` auf dem persönlichen Echo-Channel → Ring-UI + Klingelton über `SoundLibrary` (neuer Sound `incoming-call`, geloopt). **Cross-Tab-Dedup** über das bestehende `BroadcastChannel('railtime-app-presence')`-Muster: `{type: 'call-ring', roomUuid}` — nur der sichtbare/gewählte Tab spielt Audio, alle Tabs zeigen das Overlay, Annehmen/Ablehnen in einem Tab sendet `call-ring-stop`.
5. **Annehmen** → `Livewire accept()` → Einladung accepted, Broadcast `CallInvitationAnswered` → Redirect `/calls/{uuid}` → `CallWindow` → Token-Fetch → `room.connect()`.
6. **Ablehnen** → status declined, Broadcast, Anrufer-UI zeigt „abgelehnt“.
7. **Anrufer legt vor Annahme auf** → `endCall()` → `LiveKit deleteRoom` → Webhook `room_finished` räumt DB auf; `CallInvitationExpired`-Broadcast stoppt das Klingeln sofort (nicht auf den Job warten).

Aufwand: **2-3 Tage**

10. Phase 7 — Live-Steuerung über Reverb

10.1 Channel

```
// routes/channels.php – Ergänzung
Broadcast::channel('call.{roomUuid}', function ($user, string $roomUuid) {
    return \App\Models\Room::where('uuid', $roomUuid)
        ->whereHas('participants', fn ($q) => $q->where('user_id', $user->id))
        ->exists();
});
// Klingeln läuft über den bestehenden Channel App.Models.User.{id} – kein neuer nötig.
```

Kein Presence-Channel für den Raum: den In-Raum-Teilnehmerstatus liefert LiveKit selbst autoritativ. `call.{roomUuid}` ist ausschließlich Seitenkanal für Moderation/Status.

10.2 Events (alle nach `ChatMessageSent-Vorbild`: `ShouldBroadcastNow` + `broadcastAs`)

Klasse (app/Events/)	Channel	broadca stAs	Payload
<code>CallInvitationSent</code>	<code>App.Models.User.{inviteeId}</code>	<code>call.invited</code>	<code>roomUuid, roomName, callerId, callerName, chatId, expiresAt</code>
<code>CallInvitationAnswered</code>	<code>App.Models.User.{inviterId} + call.{uuid}</code>	<code>call.answered</code>	<code>inviteId, status accepted/declined</code>
<code>CallStarted</code>	<code>chat.{chatId}</code> (bei Chat-Anker)	<code>call.started</code>	<code>roomUuid</code> — ChatBox zeigt Beitritts-Banner
<code>CallEnded</code>	<code>call.{uuid} + chat.{chatId}</code>	<code>call.ended</code>	<code>roomUuid, reason</code>

Klasse (app/Events/)	Channel	broadcastAs	Payload
CallModerationActioned	call.{uuid}	call.moderated	targetUserId, action mute/unmute/remove/role
CallInvitationExpired	App.Models.User.{inviteeId}	call.missed	roomId — stoppt Klingeln in offenen Tabs

10.3 LiveKit-Webhooks (DB-Wahrheit)

- Route POST /webhooks/livekit (**außerhalb** der Auth-Gruppe), **CSRF-exempt** über den im Projekt vorhandenen Mechanismus (klassisches app/Http/Kernel-Layout → VerifyCsrfToken::except).
- app/Http/Controllers/Webhooks/LiveKitWebhookController.php: Signaturprüfung über WebhookReceiver des SDK (JWT im Authorization-Header, signiert mit demselben Key-Paar) — bei Fehler 403; sonst schnell 200 antworten und an RoomLifecycleService übergeben.
- Konsumierte Events → idempotente Übergänge:

Webhook-Event	Wirkung
room_started	status active, started_at
room_finished	status ended, ended_at, offene Einladungen → missed, Broadcast CallEnded
participant_joined	Teilnehmer connection=joined, joined_at (Match über livekit_identity)
participant_left	connection=left, left_at

LiveKit wiederholt Webhooks — jeder Übergang muss guard-geprüft sein. Damit ist die DB selbstheilend, auch wenn ein Tab abstürzt oder Reverb ausfällt.

10.4 Moderation (server-autoritativ)

Livewire-Aktionen im CallWindow → Room::canModerate(\$user) → LiveKitService:

Aktion	Server-API (RoomServiceClient)	Zusätzlich
Stummschalten	mutePublishedTrack	Broadcast CallModerationActioned (Toast „Du wurdest stummgeschaltet“)
Entfernen	removeParticipant	Teilnehmerzeile connection=disconnected
Rolle ändern („nur zuhören“ live erzwingen)	updateParticipant mit neuen Grants (canPublish-Toggle)	Rollen-Spalte aktualisieren

Durchsetzung geschieht immer an der SFU — Reverb spiegelt nur für die UX, nie für die Sicherheit.

Aufwand: **1,5-2 Tage**

11. Phase 8 — Test und Absicherung

11.1 Testmatrix

Netze:

Szenario	Erwartung
Büro-LAN (UDP frei)	Direkte UDP-Medien 50000-60000
Firmen-Firewall, UDP blockiert	Automatischer Fallback TCP 7881
Nur 443 erlaubt (strengste FW)	TURN-TLS; testweise erzwingen mit iceTransportPolicy: 'relay' im Client
Mobilfunk LTE/5G	Verbindung + Qualität

Szenario	Erwartung
Handover WLAN ↔ LTE	LiveKit-Reconnect greift, Gespräch überlebt
Zug-WLAN (hoher Paketverlust)	Degradation ohne Abbruch — der markengerechte RailTime-Testfall

Geräte: Chrome/Edge/Firefox Desktop, Safari macOS, iOS-Safari-PWA (Autoplay-/Berechtigungs-Eigenheiten; Web-Push-Klingeln ab iOS 16.4 nur bei installierter PWA), Android-Chrome-PWA; Verhalten im Hintergrund-Tab.

Last: Gruppenanrufe mit 2 / 4 / 8 / 15 Teilnehmern; CPU/Bandbreite der Media-VM beobachten; `livekit-cli load-test` existiert genau dafür.

Chaos: LiveKit-Container mitten im Anruf killen (Client-Reconnect + Webhook-getriebene DB-Heilung); Reverb killen (Anruf muss weiterlaufen — Medien sind unabhängig); Rejoin mit abgelaufenem Token.

11.2 Betrieb & Monitoring

- LiveKit-Prometheus-Endpoint (`prometheus_port: 6789`, nur für localhost/Monitoring-Host freigegeben) oder minimal: Uptime-Check auf `https://livekit.rail-time.de/` + `railtime:livekit-check` im Scheduler mit Fehlerbenachrichtigung.
- coturn: `log-file` + `logrotate`.
- Backups: Räume/Teilnehmer/Einladungen liegen im bestehenden DB-Backup; die Media-VM ist zustandslos (nur `/opt/railtime-media/-Configs` sichern; keine Aufzeichnungen in v1).
- Zertifikats-Renewals (Caddy automatisch; `certbot-Hook` für coturn).

11.3 Risiken

Risiko	Einordnung
iOS-Push-Klingeln unzuverlässig	Reverb-Klingeln ist der Primärweg; Push ist Weck-Fallback mit TTL = Ring-Fenster. PWA-Installation nötig.
Database-Queue-Latenz beim Ring-Timeout	±Sekunden, akzeptabel; bei Bedarf später Redis-Queue (Redis liegt bereits vor).
Plesk-Nginx proxied versehentlich Medien	Verbindlich: nur Signalisierung über Plesk; Medien ausschließlich Media-VM.
DSGVO/AVV	Bei externem Hoster der Media-VM: Auftragsverarbeitungsvertrag ergänzen.

Aufwand: **2-3 Tage**

12. Aufwand und Umsetzungsreihenfolge

Phase	Inhalt	Aufwand
1	Grundinfrastruktur prüfen, Entscheidungen D1–D4	0,5 Tage
2	coturn (Media-VM, DNS, Zertifikate, Firewall)	1 Tag
3	LiveKit-Server (Compose, Caddy, Keys, Webhook, Artisan-Checks)	1–1,5 Tage
4	Raumverwaltung (Migrationen, Models, RBAC, LifecycleService)	1,5 Tage
5	Tokens & Beitritt (SDK, Service, Controller, CallWindow, calls.js)	2–3 Tage
6	Einladungssystem (Service, Events, Push, Job, Overlay, Sound)	2–3 Tage
7	Live-Steuerung (Webhooks, Channel, Events, Moderation)	1,5–2 Tage
8	Test & Absicherung (Matrix, Last, Chaos, Monitoring)	2–3 Tage
Summe		~12-15 Entwicklertage

De-Risking: Die Laravel-Phasen 4-5 können **parallel** zur VM-Bereitstellung (Phasen 2-3) gegen das kostenlose LiveKit-Cloud-Kontingent entwickelt werden — nur `LIVEKIT_URL/Keys` unterscheiden sich.

Anhang A — Datei-für-Datei-Änderungsliste (Referenz für die Implementierung)

Neue Dateien (Repo):

Pfad	Inhalt
config/livekit.php	Konfiguration (Abschnitt 8.4)
database/migrations/2026_07_29_000001_create_rooms_table.php	rooms
database/migrations/2026_07_29_000002_create_room_participants_table.php	room_participants
database/migrations/2026_07_29_000003_create_room_invitations_table.php	room_invitations
app/Models/Room.php, app/Models/RoomParticipant.php, app/Models/RoomInvitation.php	Models (7.2)
app/Services/Calls/LiveKitService.php	Token-Mint + RoomServiceClient-Wrapper
app/Services/Calls/RoomLifecycleService.php	Statusübergänge (einziger Schreiber)
app/Services/Calls/CallInvitationService.php	Einladungs-Flow
app/Http/Controllers/Calls/CallTokenController.php	Token-Endpoint
app/Http/Controllers/Webhooks/LiveKitWebhookController.php	Webhook-Empfänger
app/Livewire/Calls/CallWindow.php (+ View)	Anruf-Fenster
app/Livewire/Calls/IncomingCallOverlay.php (+ View)	Klingel-Overlay (Layout-mounted)
app/Events/CallInvitationSent.php, CallInvitationAnswered.php, CallStarted.php, CallEnded.php, CallModerationActioned.php, CallInvitationExpired.php	Broadcast-Events (10.2)
app/Jobs/ExpireCallInvitation.php	Ring-Timeout
app/Notifications/IncomingCallNotification.php	Web-Push-Klingeln
app/Console/Commands/GenerateLiveKitCredentials.php	railtime:livekit-keys
app/Console/Commands/CheckLiveKitConnection.php	railtime:livekit-check
resources/js/calls.js	Alpine-Modul um livekit-client (eigener Vite-Input)

Zu ändernde Dateien (Repo):

Pfad	Änderung
app/Support/Rbac/RbacCatalog.php	Gruppe „Anrufe“: <code>calls.start</code> , <code>calls.join</code> , <code>calls.moderate</code>
routes/web.php	Token-Route, CallWindow-Route, Webhook-Route
routes/channels.php	Channel <code>call.{roomId}</code>
app/Support/Push/PushCategory.php	Case Calls = 'calls' (+ Lang-Key)
app/Models/User.php	<code>roomParticipations()</code> , <code>activeCall()</code>
app/Models/Chat.php	<code>activeRoom()</code>
app/Livewire/ChatBox.php	Anruf-Button / Beitritts-Banner
app/Support/Sound/SoundLibrary.php	Sound incoming-call (Loop)

Pfad	Änderung
vite.config.js	Input resources/js/calls.js
resources/js/app.js	Echo-Listener fürs Klingel-Overlay (leichtgewichtig, ohne livekit-client)
app/Http/Middleware/VerifyCsrfToken.php	Ausnahme webhooks/livekit
.env.example	LIVEKIT_*, CALL_RING_TIMEOUT, TURN_* (8.4)
composer.json / package.json	agence104/livekit-server-sdk / livekit-client

Neue Dateien (Media-VM, nicht im Repo — als Infra-Notiz in docs/ ablegen):

/opt/railtime-media/{docker-compose.yml, livekit.yaml, Caddyfile, turnserver.conf},
/etc/systemd/system/railtime-media.service.

Anhang B — Plan B: LiveKit auf dem Plesk-Host (falls keine zweite VM)

Docker Engine auf der Plesk-VM installieren; LiveKit im Host-Netzwerk; TLS für livekit.rail-time.de terminiert der **Plesk-Nginx** („Zusätzliche Nginx-Direktiven" → Proxy auf 127.0.0.1:7880, gleiches Muster wie der bestehende Reverb-WSS-Proxy).

Risiken, die diesen Weg zur zweiten Wahl machen:

- Port 443 für TURN-TLS ist **unmöglich** (Plesk belegt ihn) → TURN-TLS nur auf 5349, was die strengsten Firewalls blockieren.
- Die Plesk-Firewall (fail2ban/firewalld) muss die UDP-Range 50000–60000 dauerhaft freihalten — Plesk-Updates können Regeln zurücksetzen.
- CPU-Konkurrenz zwischen PHP-FPM und der SFU unter Last.

Anhang C — Umgebungsvariablen (Übersicht)

```
# LiveKit (Laravel .env)
LIVEKIT_URL=https://livekit.rail-time.de
LIVEKIT_WS_URL=wss://livekit.rail-time.de
LIVEKIT_API_KEY=RAILTIME_API_KEY
LIVEKIT_API_SECRET=<secret>
LIVEKIT_TOKEN_TTL=21600
CALL_RING_TIMEOUT=45

# TURN-Umschaltung (D4)
LIVEKIT_TURN_MODE=embedded      # embedded | coturn
TURN_URL=                       # turns:turn.rail-time.de:5349 (nur bei coturn)
TURN_USERNAME=
TURN_CREDENTIAL=
```

Ende des Dokuments. Markdown-Quelle = maschinenlesbare Referenz; PDF inhaltsgleich.